Control structures

Conditional statements and loops

Sequential execution is not enough

- Previously, our code executed instructions line after line, in order
- But sometimes, we need to skip over some instructions, or repeat instructions
- This is where control structures, such as conditional statements and loops come into play

The if statement

- if <test>:
  <statement>
- <test> needs to be an expression that evaluates to true or false
  - If <test> expression is true, then <statement> is executed
  - If <test> expression is false, then <statement> is skipped
- An example in conditionals.py
- What if you want multiple statements to be executed when <test> is true and skipped when <test> is false?

Compound statements

- Almost every programming language has some way to group multiple instructions together to form a compound statement
- Python is one of very few languages that uses indentation to create compound statements
- In python documentation, such compound statements are referred to as a suite
- A suite ends right before the first line that is not as indented as the lines of the suite

Nested if statement example

- myVar=15
  if myVar>0:
      print(“myVar is positive”)
      if myVar>20:
          print(“inner statement 1”)
      print(“Between inner conditions”)
      if myVar<20:
          print(“inner statement 2”)
      print(“End of outer condition”)
  print(“After outer condition”)

The else statement

- if <test>:
  <statement1>
  else:
  <statement2>
- If <test> is true, then <statement1> is executed and <statement2> is skipped
- If <test> is false, then <statement1> is skipped and <statement2> is executed
- Edit conditionals.py to use an else statement instead of two ifs
- Does that change the behavior of what was the two ifs?

The elif statement

- if <test1>:
  <statement1>
  elif <test2>:
  <statement2>
  elif <test3>:
  <statement3>
  else:
  <final statement>
- If <test1> is true, execute <statement1> only
- If <test1> is false and <test2> is true, execute <statement2> only
- If <test1> and <test2> are false and <test3> is true, execute <statement3> only
- If all tests are false, execute <final statement> only (else and <final statement> are optional)

Loops

- Sometimes we don’t want to selectively skip over instructions, but instead want to repeat instructions
- while <test>:
    <statement>
- Like an if statement, if <test> is false skip <statement>, and if <test> is true execute <statement>.
- But after executing <statement>, check <test> again and if still true, execute <statement> again and check again …….
- Watch out for infinite loops!

Number guessing game

- We now know almost enough to create a number guessing game
- But we will need a way to generate random numbers
- import random
- Imported libraries are a useful feature of python
- “pip list” in Codespaces terminal to see what python libraries are already installed
- random.randrange(1,10)